More Information on Code Reviews

Consistent coding standards are essential for large-scale projects, ensuring readability and maintainability. Google's style guides stand as prominent examples of how such norms can be established and adhered to across diverse teams. Code reviews are also essential in order to produce quality code. This reading delves into the principles of code review strategies and the significance of style guides, shedding light on their impact on software development practices and outcomes. You'll explore Google's style guides, learn about diverse code review strategies, and gain insights into the significance of pull request reviews.

Google style guides

Every major open-source project includes a style guide, which is a set of norms for writing code for that project. When all of the code in a huge codebase is written in the same manner, it is considerably simpler to understand.

You can find the project and style guide for Google code here.

Code review

Code review, also referred to as peer code review, is the deliberate and methodical gathering of other programmers to examine each other's code for errors. Code review can speed up and simplify the software development process, unlike other techniques. Peer reviews also save time and money, especially by catching the kinds of defects that could sneak through testing, production, and into the laptops of end users.

Common code review strategies

Pair programming

This method of building software places engineers side-by-side, working on the same code together. Pair programming is one of the defining characteristics of Extreme

Programming (XP). It seems to integrate code review directly into the programming process and is a fantastic technique for senior engineers to mentor junior team members. However, different approaches to code review might offer greater objectivity because writers and even co-authors often become too familiar with their own work. Compared to other approaches, pair programming can require more staff and time resources.

The email thread

With the email thread strategy, a file is sent to the appropriate coworkers through email as soon as a particular piece of code is prepared for review, so they can individually review it. Although this method can be more adaptable and flexible than more conventional approaches, an email thread of suggestions and divergent opinions can become confusing very quickly, leaving the original coder to sort through it all.

Over the shoulder

One of the oldest, simplest, and most natural ways to participate in peer code review is the over-the-shoulder technique, which is more comfortable for most engineers than XP's pair programming. When your code is complete, ask a coworker to evaluate it while you explain why you created it that way.

Tool assisted

Software-based code review tools, some of which are browser-based or seamlessly integrate into a range of common IDE and SCM development frameworks, are the final form of code review. Software tools enable reviews to happen asynchronously and non-locally, issuing notifications to the original coder when new reviews come in. The tools keep the review process moving efficiently with no meetings and no one having to leave their desks to contribute. Some technologies can also produce vital usage statistics that provide the audit trials and review metrics required for process improvement and compliance reporting.

Pull request reviews

A pull request (PR) is a procedure where new code is examined before it is merged to create a branch or master branch in GitHub. Before a pull request is merged, reviews give contributors the opportunity to comment on the modifications suggested in the request, accept the changes, or ask for additional changes. Administrators of the repository can mandate that each pull request be accepted before it is merged.

Anyone with read access can review and comment on the changes proposed in a pull request once it has been opened. Additionally, you can make precise changes to code lines that the author can implement right from the pull request. If you are interested in learning more about reviewing proposed changes in a pull request, click here.

Five tips for pull request reviews

Some of the considerations you should have with pull request reviews are:

1. **Be selective with reviewers:** It's important to select a reasonable number of reviewers for a pull request. Adding too many reviewers can lead to inefficient use of resources, as too many people reviewing the same code may not be productive.
2. **Timely reviews:** Ideally, reviews should be completed within two hours of the pull request being submitted. Delays in reviews can lead to context switching and hinder overall productivity.
3. **Constructive feedback:** Feedback should be constructive and explain what needs to be changed and, more importantly, why those changes are suggested. Friendly and non-accusatory language fosters a positive and collaborative atmosphere.
4. **Detailed pull request description:** The pull request should include a detailed description that covers the changes made in the feature branch compared to the development branch, prerequisites, usage instructions, design changes with comparisons to mockups, and any additional notes that reviewers should be aware of. This information ensures that reviewers have a comprehensive understanding of the changes.
5. **Interactive rebasings:** Interactive Rebasings allow developers to modify individual commits without cluttering the commit history with redundant or unrelated changes.

Keeping commits clean and relevant contributes to a more organized and maintainable codebase.

Key takeaways:

- **Importance of consistent coding standards:** Maintaining uniform coding standards ensures readability and ease of maintenance. Google's style guides serve as prime examples of establishing and adhering to such norms across diverse teams.
- **Role of code reviews:** Code reviews, or peer code reviews, involve organized examination by fellow programmers, speeding up development and catching defects that might bypass testing, saving time and resources.
- **Diverse code review strategies:** Pair programming, email threads, over-the-shoulder evaluations, and tool-assisted review strategies offer different levels of collaboration and objectivity, catering to different project needs.
- **Pull request reviews:** Pull request reviews provide an opportunity for collaborative examination of new code before integration. Accessible to those with read access, PR reviews enable inclusive feedback and ensure code quality through timely and constructive comments.